

PSSE Script for SVG-INV

```
# List of buses in PSSE where the plant transformers (SVGs) are connected.
# These are the high-voltage connection points of your plant to the grid.
# In this case, two plant transformer buses: 3001 and 3002.
PLANT_BUSES = [3001, 3002]

# The bus number that the plant voltage regulator is controlling (regulating bus).
# PSSE uses this bus to measure voltage and adjust reactive output accordingly.
# Here bus 1000 is the Point of Interconnection (POI) or collector bus being regulated.
PLANT_IREG = 1000

# Maximum number of Q correction attempts allowed inside each outer iteration.
# Each attempt dispatches reactive power and re-solves the load flow.
# 20 attempts is usually enough for Q to converge to the target value.
MAX_Q_INNER = 20

# Proportional gain for the Active Power (P) control loop.
# A value of 1.0 means: apply 100% of the P error as a correction each iteration.
# Lower values (e.g. 0.5) give slower but more stable convergence.
KP_P = 1.0

# Proportional gain for the Reactive Power (Q) control loop.
# A value of 1.0 means: apply 100% of the Q error as a correction each iteration.
# Same concept as KP_P but for reactive power.
KQ_Q = 1.0

# Minimum reactive power (Mvar) that inverters are allowed to produce.
# Negative value means inverters CAN absorb reactive power from the grid.
# Set to -999.0 so there is effectively no floor – inverters can absorb as much
# Q as needed. This is critical when the target POI Q = 0 Mvar, because the
# network itself may be generating excess Q that inverters need to absorb.
INV_Q_MIN_FLOOR = -999.0

# Starting reactive power setpoint for SVGs at the beginning of the simulation.
# 0.0 Mvar means SVGs start neutral (neither injecting nor absorbing Q).
# The control loop will then drive them toward whatever Q is needed.
SVG_Q_INIT = 0.0

# Reactive power capability factor for inverters, expressed as a fraction of MBASE.
# 0.436 corresponds to a power factor of approximately 0.916 (cos(arctan(0.436))).
# Example: a 10 MVA inverter (MBASE=10) can produce/absorb up to 4.36 Mvar.
# This is used when PSSE has no Q limits defined for an inverter in the case file.
INV_Q_FACTOR = 0.436

# Dictionary to manually override the MBASE (rated MVA) of specific machines.
# Format: {(bus_number, machine_id): new_mbase_value}
# Empty {} means no overrides – use whatever MBASE is in the PSSE case file.
# Use this if the case file has incorrect or placeholder MBASE values.
MBASE_OVERRIDE = {}

# Dictionary to manually override the Q limits (Qmin, Qmax) of specific machines.
# Format: {(bus_number, machine_id): (qmin_value, qmax_value)}
# Empty {} means no overrides – use limits from the PSSE case file.
# Use this if a machine has wrong or zero Q limits in the case file.
Q_LIMIT_OVERRIDE = {}

# =====
# BUS CONFIGURATION
# =====

# Bus number of the Point of Interconnection (POI) – the metering point
# where your plant connects to the utility grid. All P and Q targets
# are measured at this bus.
POI_BUS = 999

# Bus number range for all solar inverters in the PSSE case file.
# Every machine whose bus number falls between these two values
# is treated as a solar inverter by the controller.
# Example: buses 11001, 11002, ... 11136 are all inverter buses.
INV_START = 11001
```

```

INV_END    = 11136

# List of buses where the SVG (Static Var Generator) machines are connected.
# SVGs are the primary reactive power (Q) control devices in this plant.
# They are dispatched first before inverters are asked to provide Q.
SVG_BUSES = [3001, 3002]

# List of buses where the main plant power transformers are connected.
# In this small network these are the same as SVG_BUSES because the
# SVGs sit at the plant transformer high-voltage terminals.
PLANT_BUSES = [3001, 3002]

# The bus number that the plant voltage regulator monitors and controls.
# PSSE adjusts reactive output to keep this bus voltage at the scheduled value.
# Typically this is the POI bus or the collector bus (bus 1000 here).
PLANT_IREG = 1000

# =====
# VOLTAGE SETPOINT
# =====

# Initial voltage schedule (in per-unit) applied to the plant transformer buses
# at the start of the simulation before the control loop begins.
# 1.05 pu = 5% above nominal voltage.
VSET_INIT = 1.0500

# Minimum allowed voltage schedule (per-unit).
# The controller will never set plant voltage below this value.
VSET_MIN = 0.95

# Maximum allowed voltage schedule (per-unit).
# The controller will never set plant voltage above this value.
VSET_MAX = 1.10

# =====
# POWER TARGETS AT POI
# =====

# Target active power (MW) to be exported at the POI.
# Negative sign follows PSSE generator convention:
# negative = power flowing OUT of the plant INTO the grid.
# So -300.0 means the plant should export 300 MW to the grid.
TARGET_POI_P = -300.0

# Target reactive power (Mvar) at the POI.
# 0.0 means unity power factor – the plant should neither
# inject nor absorb reactive power at the metering point.
TARGET_POI_Q = 0.0

# =====
# CONVERGENCE TOLERANCES
# =====

# Acceptable error band for active power (MW).
# If |actual POI P - target POI P| is within 0.5 MW, P is considered converged.
PTOL = 0.5

# Acceptable error band for reactive power (Mvar).
# If |actual POI Q - target POI Q| is within 0.25 Mvar, Q is considered converged.
QTOL = 0.25

# =====
# ITERATION LIMITS
# =====

# Maximum number of outer control loop iterations.
# Each outer iteration runs one P correction followed by one full Q inner loop.
# Increase this if P and Q are not converging within 10 iterations.
MAX_ITER = 10

# Maximum number of Q correction attempts inside each outer iteration.
# Each attempt dispatches reactive power to SVGs/inverters and re-solves load flow.
# 20 is usually sufficient; increase only if Q oscillates without converging.
MAX_Q_INNER = 20

```

```

# =====
# CONTROL GAINS
# =====

# Proportional gain for the Active Power (P) control loop.
# 1.0 = apply 100% of the P error as correction in one step.
# Reduce to 0.5 if P overshoots and oscillates between iterations.
KP_P = 1.0

# Proportional gain for the Reactive Power (Q) control loop.
# 1.0 = apply 100% of the Q error as correction in one inner iteration.
# Reduce to 0.5 if Q oscillates or SVGs are hunting (swinging back and forth).
KQ_Q = 1.0

# =====
# INVERTER ACTIVE POWER LIMITS
# =====

# Minimum active power output per inverter (MW).
# 0.0 means inverters cannot run in reverse (no motoring).
# This is a physical limit – solar inverters cannot consume real power.
MIN_INV_P = 0.0

# Maximum active power output per inverter (MW).
# Each individual inverter is capped at 10 MW regardless of its MBASE.
# Adjust this to match the rated output of a single inverter in your plant.
MAX_INV_P = 10.0

# =====
# INVERTER REACTIVE POWER FLOOR
# =====

# Minimum reactive power (Mvar) that any inverter is allowed to produce.
# Negative = inverter is allowed to ABSORB reactive power from the grid.
# Set to -999.0 (effectively no floor) so inverters can absorb as much Q
# as needed. This is essential when TARGET_POI_Q = 0.0 because the network
# cables and transformers naturally generate capacitive Q that must be absorbed.
# If you set this to 0.0, inverters can only inject Q and will never reach Q=0.
INV_Q_MIN_FLOOR = -999.0

# =====
# SVG INITIALISATION
# =====

# Reactive power setpoint (Mvar) applied to SVGs at the very start,
# before the control loop begins. 0.0 = start neutral.
# The control loop will then drive SVGs to whatever Q is needed.
# Do NOT use a large value here (e.g. 50 Mvar) as it pushes the
# system far from the target before control even starts.
SVG_Q_INIT = 0.0

# =====
# INVERTER Q CAPABILITY FACTOR
# =====

# Reactive power capability of inverters expressed as a fraction of their
# rated MVA (MBASE). 0.436 corresponds to ~power factor 0.916.
# Example: a 10 MVA inverter can produce or absorb up to 4.36 Mvar.
# This factor is ONLY used as a fallback when the PSSE case file has
# zero Q limits (Qmin=0, Qmax=0) defined for an inverter machine.
# If your case file already has correct Q limits, this has no effect.
INV_Q_FACTOR = 0.436

# =====
# MANUAL OVERRIDES (leave empty unless case file has errors)
# =====

# Dictionary to manually override the rated MVA (MBASE) of specific machines.
# Format: {(bus_number, "machine_id"): new_mbase_in_MVA}
# Example: {(11001, "1"): 12.5} sets inverter at bus 11001 to 12.5 MVA.
# Leave empty {} to use whatever MBASE values are in the PSSE case file.
MBASE_OVERRIDE = {}

```

```

# Dictionary to manually override the Q limits of specific machines.
# Format: {(bus_number, "machine_id"): (qmin_in_Mvar, qmax_in_Mvar)}
# Example: {(3001, "1"): (-50.0, 50.0)} sets SVG at bus 3001 to ±50 Mvar.
# Leave empty {} to use Q limits from the PSSE case file.
# Use this if a machine has incorrect or zero Q limits in the case file.
Q_LIMIT_OVERRIDE = {}

import psspy
import redirect
import math
import logging

logging.basicConfig(level=logging.WARNING, format="%(levelname)s: %(message)s")

try:
    redirect.psse2py()
except Exception as exc:
    logging.warning("redirect.psse2py() failed (non-fatal): %s", exc)

# =====
# USER INPUT – update bus ranges to match your small network
# =====

POI_BUS      = 999
INV_START    = 11001
INV_END      = 11136
SVG_BUSES    = [3001, 3002]  # replaces PTR_BUSES – your SVG machine buses
PLANT_BUSES  = [3001, 3002]
PLANT_IREG   = 1000

VSET_INIT    = 1.0500
VSET_MIN     = 0.95
VSET_MAX     = 1.10

TARGET_POI_P = -300.0
TARGET_POI_Q = 0.0

PTOL = 0.5
QTOL = 0.25

MAX_ITER     = 10
MAX_Q_INNER  = 20

KP_P = 1.0
KQ_Q = 1.0

MIN_INV_P = 0.0
MAX_INV_P = 10.0

# Allow inverters to absorb Q (negative) – critical for Q=0 target
INV_Q_MIN_FL00R = -999.0

SVG_Q_INIT = 0.0          # start SVGs at 0 Mvar, let control loop drive them
INV_Q_FACTOR = 0.436

MBASE_OVERRIDE = {}
Q_LIMIT_OVERRIDE = {}

_i = psspy.getdefaultint()
_f = psspy.getdefaultreal()
_s = psspy.getdefaultchar()

# =====
# EXCEPTIONS
# =====

class PSSEError(RuntimeError): pass
class ConvergenceError(RuntimeError): pass
class DataError(ValueError): pass

# =====
# HELPERS

```

```

# =====

def clamp(x, xmin, xmax):
    return max(xmin, min(x, xmax))

def solve():
    ierr = psspy.fns1([0, 0, 0, 0, 1, 0, 3, 0])
    if ierr != 0:
        raise ConvergenceError("Load flow failed ierr={}".format(ierr))

def get_bus_code(bus):
    ierr, code = psspy.busint(bus, 'TYPE')
    if ierr != 0:
        raise PSSEError("busint(TYPE) failed bus={} ierr={}".format(bus, ierr))
    return code

def set_bus_code(bus, code):
    ierr = psspy.bus_chng_3(
        bus, [code, _i, _i, _i], [_f, _f, _f, _f, _f, _f, _f], _s
    )
    if ierr != 0:
        raise PSSEError("bus_chng_3 failed bus={} code={} ierr={}".format(bus, code, ierr))

def get_all_machine_data():
    ierr, bnums = psspy.amachint(-1, 4, ['NUMBER'])
    if ierr != 0:
        raise PSSEError("amachint(NUMBER) failed ierr={}".format(ierr))

    ierr, ids = psspy.amachchar(-1, 4, ['ID'])
    if ierr != 0:
        raise PSSEError("amachchar(ID) failed ierr={}".format(ierr))

    ierr, reals = psspy.amachreal(-1, 4, ['PGEN', 'QGEN', 'MBASE', 'QMAX', 'QMIN'])
    if ierr != 0:
        raise PSSEError("amachreal(multi) failed ierr={}".format(ierr))

    pgs, qgs, mbs, qmaxs, qmins = reals
    data = []
    for bus, mid, pg, qg, mbase, qmax, qmin in zip(
        bnums[0], ids[0], pgs, qgs, mbs, qmaxs, qmins):
        mid = mid.strip()
        key = (bus, mid)

        if key in MBASE_OVERRIDE:
            mbase = MBASE_OVERRIDE[key]

        if key in Q_LIMIT_OVERRIDE:
            qmin, qmax = Q_LIMIT_OVERRIDE[key]
        elif INV_START <= bus <= INV_END and abs(qmin) < 1e-9 and abs(qmax) < 1e-9:
            qmax = mbase * INV_Q_FACTOR
            qmin = -mbase * INV_Q_FACTOR

        data.append({
            "bus": bus, "id": mid,
            "pg": pg, "qg": qg,
            "mbase": mbase, "qmin": qmin, "qmax": qmax
        })
    return data

def get_inverter_machines(all_data):
    return [m for m in all_data if INV_START <= m["bus"] <= INV_END]

def get_svg_machines(all_data):
    # SVG buses only – no UST, no WTG
    return [m for m in all_data if m["bus"] in SVG_BUSES]

def split_machines(all_data):

```

```

    inv = get_inverter_machines(all_data)
    svg = get_svg_machines(all_data)
    return inv, svg

def get_effective_q_limits(m):
    qcap = math.sqrt(max(m["mbase"]**2 - m["pg"]**2, 0.0))
    qmin_eff = max(m["qmin"], -qcap)
    qmax_eff = min(m["qmax"], qcap)

    if qmin_eff > qmax_eff:
        qmin_eff = qmax_eff = 0.0

    # Apply floor only for inverters
    if INV_START <= m["bus"] <= INV_END:
        qmin_eff = max(qmin_eff, INV_Q_MIN_FLOOR)

    if qmin_eff > qmax_eff:
        qmin_eff = qmax_eff = 0.0

    return qmin_eff, qmax_eff, qcap

def get_effective_p_limit(m):
    return min(MAX_INV_P, math.sqrt(max(m["mbase"]**2 - m["qg"]**2, 0.0)))

def set_machine_pg(bus, mid, new_pg):
    ierr = psspy.machine_chng_2(
        bus, mid, [_i, _i, _i, _i, _i, _i], [new_pg, _f, _f, _f, _f, _f]
    )
    if ierr != 0:
        raise PSSEError("machine_chng_2(PGEN) failed bus={ } id={!r} ierr={ }".format(bus, mid, ierr))

def set_machine_qg(bus, mid, new_qg, qmin_eff, qmax_eff):
    # Step 1: widen limits first
    ierr = psspy.machine_chng_2(
        bus, mid, [_i, _i, _i, _i, _i, _i], [_f, _f, qmax_eff, qmin_eff, _f, _f]
    )
    if ierr != 0:
        raise PSSEError("machine_chng_2(QLIM) failed bus={ } id={!r} ierr={ }".format(bus, mid, ierr))

    # Step 2: write QGEN
    ierr = psspy.machine_chng_2(
        bus, mid, [_i, _i, _i, _i, _i, _i], [_f, new_qg, _f, _f, _f, _f]
    )
    if ierr != 0:
        raise PSSEError("machine_chng_2(QGEN) failed bus={ } id={!r} ierr={ }".format(bus, mid, ierr))

    ierr, qchk = psspy.macdat(bus, mid, 'QGEN')
    if ierr == 0:
        logging.debug("Q WRITE bus=%d id=%s target=%.4f actual=%.4f", bus, mid, new_qg, qchk)

def set_q_pq_mode(machines, new_qg_map):
    """Force buses to PQ type and write Q. Returns original bus codes for later restore."""
    orig_codes = {}
    for m in machines:
        bus = m["bus"]
        if bus not in orig_codes:
            orig_codes[bus] = get_bus_code(bus)
            set_bus_code(bus, 1) # PQ mode

    for m in machines:
        key = (m["bus"], m["id"])
        if key in new_qg_map:
            new_qg, qmin_eff, qmax_eff = new_qg_map[key]
            set_machine_qg(m["bus"], m["id"], new_qg, qmin_eff, qmax_eff)

    return orig_codes

def restore_bus_codes(orig_codes):

```

```

for bus, code in orig_codes.items():
    set_bus_code(bus, code)

def enforce_mbase_limits(machines, label=''):
    violated = False
    for m in machines:
        s = math.hypot(m["pg"], m["qg"])
        if s > m["mbase"] + 1e-6:
            violated = True
            scale = m["mbase"] / s
            new_pg = clamp(m["pg"] * scale, MIN_INV_P, MAX_INV_P)
            qmin_eff, qmax_eff, _ = get_effective_q_limits(**m, "pg": new_pg)
            new_qg = clamp(m["qg"] * scale, qmin_eff, qmax_eff)
            logging.warning(
                "MBASE ENFORCE[%s]: bus %5d id %2s P %.3f->%.3f Q %.3f->%.3f "
                "S %.3f->%.3f MBASE=%.3f",
                label, m["bus"], m["id"],
                m["pg"], new_pg, m["qg"], new_qg,
                s, math.hypot(new_pg, new_qg), m["mbase"]
            )
            set_machine_pg(m["bus"], m["id"], new_pg)
            set_machine_qg(m["bus"], m["id"], new_qg, qmin_eff, qmax_eff)
    return violated

def set_plant_vs(bus, ireg, vsched):
    last_exc = None
    for fn in [psspy.plant_chng_4, psspy.plant_data_4, psspy.plant_data]:
        try:
            ierr = fn(bus, [ireg], [vsched, _f])
            if ierr == 0:
                return
            last_exc = PSSEError("{} returned ierr={}".format(fn.__name__, ierr))
        except Exception as exc:
            last_exc = exc
    raise PSSEError("Unable to set plant VS at bus {}: {}".format(bus, last_exc))

def set_all_plant_vs(vsched):
    for b in PLANT_BUSES:
        set_plant_vs(b, PLANT_IREG, vsched)

def get_bus_pu(bus):
    ierr, vpu = psspy.busdat(bus, 'PU')
    if ierr != 0:
        raise PSSEError("busdat(PU) failed bus={} ierr={}".format(bus, ierr))
    return vpu

def get_total_poi_pq():
    ierr, s = psspy.brnflo(POI_BUS, PLANT_IREG, '1')
    if ierr == 0:
        return s.real, s.imag
    try:
        ierr, s = psspy.cplxbrnflo(POI_BUS, PLANT_IREG, '1')
        if ierr == 0:
            return s.real, s.imag
    except Exception as exc:
        logging.warning("cplxbrnflo fallback failed: %s", exc)
    raise DataError("Branch flow query failed for POI bus {}".format(POI_BUS))

def dispatch_q(machines, dq_remaining):
    """Proportional Q dispatch across machines based on available headroom."""
    cap = 0.0
    headrooms = []
    for m in machines:
        qmin_eff, qmax_eff, _ = get_effective_q_limits(m)
        hr = max(
            (qmax_eff - m["qg"]) if dq_remaining > 0 else (m["qg"] - qmin_eff),
            0.0
        )

```

```

        headrooms.append((m, qmin_eff, qmax_eff, hr))
        cap += hr

dq_total = clamp(dq_remaining, 0.0, cap) if dq_remaining > 0 \
    else clamp(dq_remaining, -cap, 0.0)

qg_map = {}
for m, qmin_eff, qmax_eff, hr in headrooms:
    weight = hr / max(cap, 1e-9)
    new_qg = clamp(m["qg"] + dq_total * weight, qmin_eff, qmax_eff)
    qg_map[(m["bus"], m["id"])] = (new_qg, qmin_eff, qmax_eff)

return qg_map, dq_total

def check_inverter_violations(inv):
    s_viol = q_viol = 0
    for m in inv:
        s = math.hypot(m["pg"], m["qg"])
        qmin_eff, qmax_eff, _ = get_effective_q_limits(m)
        s_bad = s > m["mbase"] + 1e-6
        q_bad = m["qg"] < qmin_eff - 1e-6 or m["qg"] > qmax_eff + 1e-6
        if s_bad or q_bad:
            logging.warning(
                "VIOLATION bus %5d id %2s P=%3f Q=%3f S=%3f MBASE=%3f Qeff=[%3f,%3f]",
                m["bus"], m["id"], m["pg"], m["qg"], s, m["mbase"], qmin_eff, qmax_eff
            )
        s_viol += int(s_bad)
        q_viol += int(q_bad)
    return s_viol, q_viol

def print_summary(label, inv, svg, poi_p, poi_q, poi_v):
    s_bad, q_bad = check_inverter_violations(inv)
    print("\n=====")
    print(label)
    print("=====")
    print("POI P = {:.4f} MW (target {:.4f})".format(poi_p, TARGET_POI_P))
    print("POI Q = {:.4f} Mvar (target {:.4f})".format(poi_q, TARGET_POI_Q))
    print("POI V = {:.4f} pu".format(poi_v))
    print("Inv count = {}".format(len(inv)))
    print("Inv total P = {:.4f} MW".format(sum(m["pg"] for m in inv)))
    print("SVG total Q = {:.4f} Mvar".format(sum(m["qg"] for m in svg)))
    print("INV total Q = {:.4f} Mvar".format(sum(m["qg"] for m in inv)))
    print("S violations = {} | Q violations = {}".format(s_bad, q_bad))

# =====
# MAIN
# =====

print("\n#####")
print("POI P+Q CONTROLLER [SVG + INVERTER PLANT]")
print("#####")
print("Target POI P = {:.4f} MW | Target POI Q = {:.4f} Mvar".format(
    TARGET_POI_P, TARGET_POI_Q))

current_vs = clamp(VSET_INIT, VSET_MIN, VSET_MAX)
set_all_plant_vs(current_vs)
solve()

# =====
# SVG INIT – force SVGs to PQ mode and hold them there
# SVG buses are kept as PQ (type=1) for the entire control loop
# so PSSE cannot override Q via voltage regulation
# =====

all_data = get_all_machine_data()
_, svg0 = split_machines(all_data)

svg_orig_codes = {} # saved once, restored only at the very end
svg_map = {}
for m in svg0:
    qmin_eff, qmax_eff, _ = get_effective_q_limits(m)

```

```

new_qg = clamp(SVG_Q_INIT, qmin_eff, qmax_eff)
svg_map[(m["bus"], m["id"])] = (new_qg, qmin_eff, qmax_eff)
print(" SVG INIT bus {:5d}: Q set to {:.2f} Mvar".format(m["bus"], new_qg))

svg_orig_codes = set_q_pq_mode(svg0, svg_map)
solve()
# NOTE: do NOT restore SVG bus codes here – keep them in PQ mode

# =====
# INITIAL RESULT
# =====

all_data = get_all_machine_data()
inv0, svg0 = split_machines(all_data)
poi_p, poi_q = get_total_poi_pq()
poi_v = get_bus_pu(POI_BUS)
print_summary("INITIAL RESULT", inv0, svg0, poi_p, poi_q, poi_v)

# =====
# CONTROL LOOP
# =====

for itr in range(1, MAX_ITER + 1):

    solve()
    all_data = get_all_machine_data()
    inv, svg = split_machines(all_data)
    n_inv = len(inv)

    if n_inv == 0:
        raise DataError("No inverters found in bus range {} - {}".format(INV_START, INV_END))

    poi_p, poi_q = get_total_poi_pq()
    poi_v = get_bus_pu(POI_BUS)
    err_p = TARGET_POI_P - poi_p
    err_q = TARGET_POI_Q - poi_q

    print("\n-----")
    print("ITER {:2d}".format(itr))
    print("-----")
    print("POI P = {:.10.4f} MW | Target = {:.10.4f} | Err = {:.+10.4f}".format(
        poi_p, TARGET_POI_P, err_p))
    print("POI Q = {:.10.4f} Mvar | Target = {:.10.4f} | Err = {:.+10.4f}".format(
        poi_q, TARGET_POI_Q, err_q))
    print("POI V = {:.10.4f} pu | Plant VS = {:.10.6f} pu".format(poi_v, current_vs))
    print("SVG Q = {:.10.4f} Mvar | INV Q = {:.10.4f} Mvar".format(
        sum(m["qg"] for m in svg),
        sum(m["qg"] for m in inv)))

    s_bad, q_bad = check_inverter_violations(inv)
    print("S viol = {} | Q viol = {}".format(s_bad, q_bad))

    if abs(err_p) <= PTOL and abs(err_q) <= QTOL:
        print("\nTARGET ACHIEVED WITHIN TOLERANCE.")
        break

    # =====
    # P LOOP (fixed sign: +KP_P drives P toward target)
    # =====

    if abs(err_p) > PTOL:
        delta_p = KP_P * err_p / float(n_inv) # FIXED: was -KP_P
        for m in inv:
            p_upper = get_effective_p_limit(m)
            new_pg = clamp(m["pg"] + delta_p, MIN_INV_P, p_upper)
            set_machine_pg(m["bus"], m["id"], new_pg)
        solve()

    # =====
    # REFRESH AFTER P LOOP
    # =====

    all_data = get_all_machine_data()
    inv, svg = split_machines(all_data)

```

```

if enforce_mbase_limits(inv, label="after-P"):
    solve()
    all_data = get_all_machine_data()
    inv, svg = split_machines(all_data)

poi_p, poi_q = get_total_poi_pq()
err_q = TARGET_POI_Q - poi_q

# =====
# Q LOOP – SVG first, then inverters for remainder
# SVG buses are already in PQ mode – just write Q directly
# =====

for q_itr in range(1, MAX_Q_INNER + 1):

    if abs(err_q) <= QTOL:
        logging.debug("Q converged at inner iter %d (err_q=%+.6f)", q_itr - 1, err_q)
        break

    dq_remaining = KQ_Q * err_q
    all_data = get_all_machine_data()
    inv, svg = split_machines(all_data)

    # Priority 1: SVG
    svg_map, dq_svg = dispatch_q(svg, dq_remaining)
    dq_remaining -= dq_svg
    print(" Q[{}] SVG dispatch={:.4f} Mvar remaining={:.4f}".format(
        q_itr, dq_svg, dq_remaining))

    # Priority 2: Inverters (only if SVG headroom exhausted)
    inv_map = {}
    if abs(dq_remaining) > QTOL:
        inv_map, dq_inv = dispatch_q(inv, dq_remaining)
        print(" Q[{}] INV dispatch={:.4f} Mvar".format(q_itr, dq_inv))

    # Write SVG Q – buses already in PQ mode, no mode switch needed
    for m in svg:
        key = (m["bus"], m["id"])
        if key in svg_map:
            new_qg, qmin_eff, qmax_eff = svg_map[key]
            set_machine_qg(m["bus"], m["id"], new_qg, qmin_eff, qmax_eff)

    # Write INV Q – temporarily switch to PQ, solve, restore
    if inv_map:
        inv_restore = set_q_pq_mode(inv, inv_map)
        solve()
        restore_bus_codes(inv_restore)
        solve()
    else:
        solve()

    all_data = get_all_machine_data()
    inv, svg = split_machines(all_data)

    if enforce_mbase_limits(inv, label="q-inner-{}".format(q_itr)):
        solve()
        all_data = get_all_machine_data()
        inv, svg = split_machines(all_data)

    poi_p, poi_q = get_total_poi_pq()
    err_q = TARGET_POI_Q - poi_q
    print(" Q[{}] POI Q={:.4f} Mvar err_q={:.4f}".format(q_itr, poi_q, err_q))

else:
    logging.warning("Q inner loop reached max %d iters (err_q=%+.6f)", MAX_Q_INNER, err_q)

if enforce_mbase_limits(inv, label="after-Q"):
    solve()
    all_data = get_all_machine_data()
    inv, svg = split_machines(all_data)

# =====
# RESTORE SVG BUS CODES before final solve

```

```

# =====

restore_bus_codes(svg_orig_codes)

# =====
# FINAL
# =====

solve()
all_data = get_all_machine_data()
inv_f, svg_f = split_machines(all_data)

if enforce_mbase_limits(inv_f, label="final"):
    solve()
    all_data = get_all_machine_data()
    inv_f, svg_f = split_machines(all_data)

final_p, final_q = get_total_poi_pq()
final_v = get_bus_pu(POI_BUS)
final_s = math.hypot(final_p, final_q)
pf_a = abs(final_p) / final_s if final_s > 1e-9 else 1.0

print("\n=====")
print("FINAL RESULT")
print("=====")
print("POI P = {:.10.4f} MW | Target = {:.10.4f} | Err = {:.+10.4f}".format(
    final_p, TARGET_POI_P, TARGET_POI_P - final_p))
print("POI Q = {:.10.4f} Mvar | Target = {:.10.4f} | Err = {:.+10.4f}".format(
    final_q, TARGET_POI_Q, TARGET_POI_Q - final_q))
print("POI V = {:.10.4f} pu".format(final_v))
print("Power factor = {:.4f}".format(pf_a))
print("Inv count = {}".format(len(inv_f)))
print("Inv total P = {:.10.4f} MW".format(sum(m["pg"] for m in inv_f)))
print("SVG total Q = {:.10.4f} Mvar".format(sum(m["qg"] for m in svg_f)))
print("INV total Q = {:.10.4f} Mvar".format(sum(m["qg"] for m in inv_f)))

s_bad, q_bad = check_inverter_violations(inv_f)
print("Inverter violations: S={:d} Q={:d}".format(s_bad, q_bad))
solve()
all_data = get_all_machine_data()
inv_f, svg_f = split_machines(all_data)

if enforce_mbase_limits(inv_f, label="final"):
    solve()
    all_data = get_all_machine_data()
    inv_f, svg_f = split_machines(all_data)

final_p, final_q = get_total_poi_pq()
final_v = get_bus_pu(POI_BUS)
final_s = math.hypot(final_p, final_q)
pf_a = abs(final_p) / final_s if final_s > 1e-9 else 1.0

print("\n=====")
print("FINAL RESULT")
print("=====")
print("POI P = {:.10.4f} MW | Target = {:.10.4f} | Err = {:.+10.4f}".format(
    final_p, TARGET_POI_P, TARGET_POI_P - final_p))
print("POI Q = {:.10.4f} Mvar | Target = {:.10.4f} | Err = {:.+10.4f}".format(
    final_q, TARGET_POI_Q, TARGET_POI_Q - final_q))
print("POI V = {:.10.4f} pu".format(final_v))
print("Power factor = {:.4f}".format(pf_a))
print("Inv count = {}".format(len(inv_f)))
print("Inv total P = {:.10.4f} MW".format(sum(m["pg"] for m in inv_f)))
print("SVG total Q = {:.10.4f} Mvar".format(sum(m["qg"] for m in svg_f)))
print("INV total Q = {:.10.4f} Mvar".format(sum(m["qg"] for m in inv_f)))

s_bad, q_bad = check_inverter_violations(inv_f)
print("Inverter violations: S={:d} Q={:d}".format(s_bad, q_bad))

# Leading/lagging indicator
total_q = sum(m["qg"] for m in inv_f) + sum(m["qg"] for m in svg_f)
pf_sign = "lagging" if total_q >= 0 else "leading"
print("Power factor type = {}".format(pf_sign))

```

```

# SVG headroom report
print("\n--- SVG Headroom ---")
for m in svg_f:
    qmin_eff, qmax_eff, _ = get_effective_q_limits(m)
    print("  SVG bus {:5d}: Q={:8.4f} Mvar limits=[{: .4f}, {: .4f}] headroom_up={: .4f} headroom_dn={: .4f}".format(
        m["bus"], m["qg"], qmin_eff, qmax_eff,
        qmax_eff - m["qg"], m["qg"] - qmin_eff))

# Inverter headroom report
print("\n--- Inverter Summary (first 10) ---")
for m in inv_f[:10]:
    qmin_eff, qmax_eff, _ = get_effective_q_limits(m)
    s = math.hypot(m["pg"], m["qg"])
    print("  INV bus {:5d}: P={:7.4f} Q={:7.4f} S={:7.4f} MBASE={:7.4f} Qeff=[{: .4f},{: .4f}]"
        .format(
            m["bus"], m["pg"], m["qg"], s, m["mbase"], qmin_eff, qmax_eff))

print("\n--- DONE ---")

```